Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

ОТЧЁТ
по лабораторной работе №4
по дисциплине
«Информационные технологии и веб-программирование»
Тема: Управление состоянием в React-приложении

Выполнила: студентка группы 320604
Хаджинова К.А.

Проверил: Хаджинова Н.В.

Минск 2026

СОДЕРЖАНИЕ

1 ЦЕЛЬ РАБОТЫ 3
2 КРАТКОЕ ОПИСАНИЕ ПРИЛОЖЕНИЯ 3
3 ХОД РАБОТЫ 3
3.1 Состояние компонента 3
3.2 Хранение в localStorage 3
3.3 Фильтрация, сортировка и статистика 4
3.4 Дополнительные эффекты 4
3.5 Проверка вводимых значений 4
3.6 Устойчивость к чужим данным 4
3.7 Внешний вид 5
3.8 Демонстрация работы 5
3.9 Тестирование 13
ЗАКЛЮЧЕНИЕ 14
ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ 15
ПРИЛОЖЕНИЕ А Ключевые части кода 16

1 ЦЕЛЬ РАБОТЫ

Освоить управление локальным состоянием в React с помощью хуков `useState` и `useEffect`, обработку побочных эффектов и динамическое обновление интерфейса на примере списка диаграмм проекта DSLKit, веб-редактора диаграмм как кода (направление SaaS).

2 КРАТКОЕ ОПИСАНИЕ ПРИЛОЖЕНИЯ

Приложение показывает список диаграмм пользователя. У диаграммы есть название, нотация (UML, BPMN, ERD, DFD, IDEF0, сети Петри), статус «Черновик» или «Опубликована» и дата изменения. Пользователь добавляет диаграмму через форму, переименовывает её, меняет статус, удаляет, ищет по названию, фильтрует по статусу и сортирует по дате или названию. Список переживает перезагрузку страницы, потому что хранится в `localStorage`.

Проект создан на Vite командой `npm create vite@latest` с шаблоном React, тесты написаны на Vitest и React Testing Library. Запуск описан в `README.md` работы.

3 ХОД РАБОТЫ

3.1 Состояние компонента

Массив диаграмм, поле ввода названия, выбранная нотация, строка поиска, фильтр статуса и режим сортировки хранятся через `useState`. Изменения массива записаны функциональными обновлениями вида `setDiagrams((prev) => ...)`, поэтому быстрые подряд идущие действия не теряют друг друга. Поле ввода сделано контролируемым, форма отправляется и кнопкой, и клавишей Enter. Ключевые части кода приведены в приложении А.

3.2 Хранение в localStorage

Логика хранения вынесена в хук `useLocalStorage`. Начальное значение читается из `localStorage` функцией-инициализатором `useState`, поэтому при первом рендере уже видны сохранённые данные. Эффект `useEffect` с зависимостью от значения записывает массив через `JSON.stringify` при каждом изменении. Если ключа нет или JSON повреждён, берётся начальный набор из трёх диаграмм.

3.3 Фильтрация, сортировка и статистика

Функции `filterDiagrams`, `sortDiagrams` и `getStats` чистые и лежат в `utils/diagrams.js`, поэтому проверяются без отрисовки. Сортировка работает с копией массива и не изменяет состояние.

3.4 Дополнительные эффекты

Заголовок вкладки обновляется эффектом и показывает число диаграмм. Хук `useAutosaveStatus` показывает «Сохранение…» после изменения и «Сохранено» через 500 мс тишины. Это debounce: каждое новое изменение очищает прежний таймер функцией очистки эффекта.

Третий эффект имитирует загрузку с сервера. Хук `useServerLoad` при старте включает признак загрузки, через 1 секунду снимает его таймером и очищает таймер при закрытии страницы. Пока признак включён, вместо списка показывается сообщение «Загрузка диаграмм…».

3.5 Проверка вводимых значений

Форма и режим редактирования проверяют название диаграммы до сохранения. Значение из одних пробелов считается пустым и отклоняется сообщением «Введите название диаграммы». Название длиннее предела отклоняется сообщением с числом допустимых символов. Пределы длины названия и числа записей лежат в отдельном файле `src/limits.json`, а не в коде компонентов. Список ограничен 200 записями, поэтому бесконечные добавления не заполнят `localStorage`. Поле «Создана» у каждой записи показывает время создания с секундами. Если запись создана в ту же секунду, что и предыдущая, время сдвигается на секунду вперёд, поэтому у записей нет одинакового времени.

3.6 Устойчивость к чужим данным

Содержимое `localStorage` может быть изменено вручную или остаться от прежней версии. Функция `isDiagramList` принимает только список, где у каждой записи есть допустимое название, известная нотация, логический статус и корректные даты, а идентификаторы не повторяются. Если список целиком не подходит, функция `repairDiagramList` оставляет только годные записи, отбрасывает повреждённые и повторяющиеся и обрезает список до предела. Если годных записей нет, берётся начальный

набор. Названия выводятся как текст, поэтому разметка внутри названия не выполняется. Поле названия ограничено атрибутом `maxLength` и той же проверкой на сервисном уровне. Список можно фильтровать по нотации. Карточка обёрнута в `memo`, а обработчики в `useCallback`, поэтому она перерисовывается только при изменении своей записи. Идентификатор создаётся через `crypto.randomUUID`, а на адресе без защищённого контекста подставляется запасной способ. Компонент `ErrorBoundary` показывает страницу 500 в стиле сайта вместо пустого экрана при сбое отрисовки.

3.7 Внешний вид

Приложение использует те же стили и разметку, что сайт первой части: закреплённую шапку с логотипом, меню с кнопкой для телефона, переключатель темы в виде луны и солнца с подсказкой, карточки, поля форм и подвал. Светлая тема включена первой, тёмная выбирается по настройке системы или переключателем. Одна главная кнопка на экране это «Добавить».

3.8 Демонстрация работы

Начало страницы с шапкой и формой добавления показано в верхней части рисунка 1.

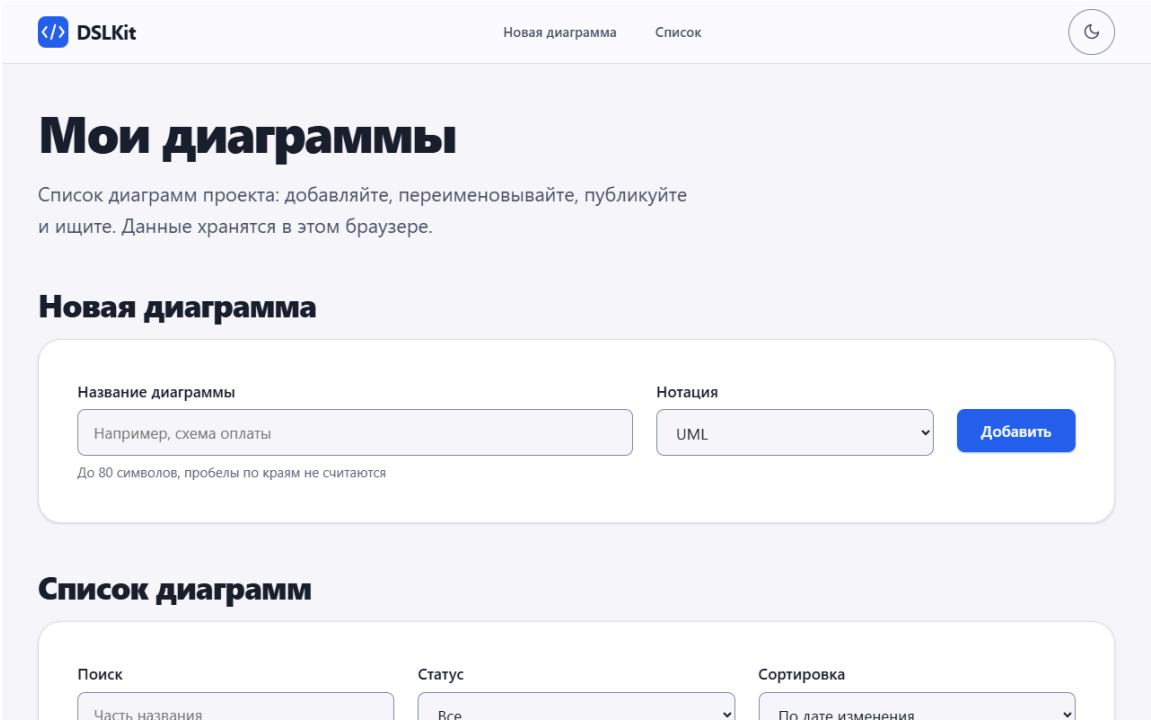


Рисунок 1 – Шапка, заголовок и форма добавления диаграммы в светлой теме

Ответ на ввод из одних пробелов виден под полем на рисунке 2.

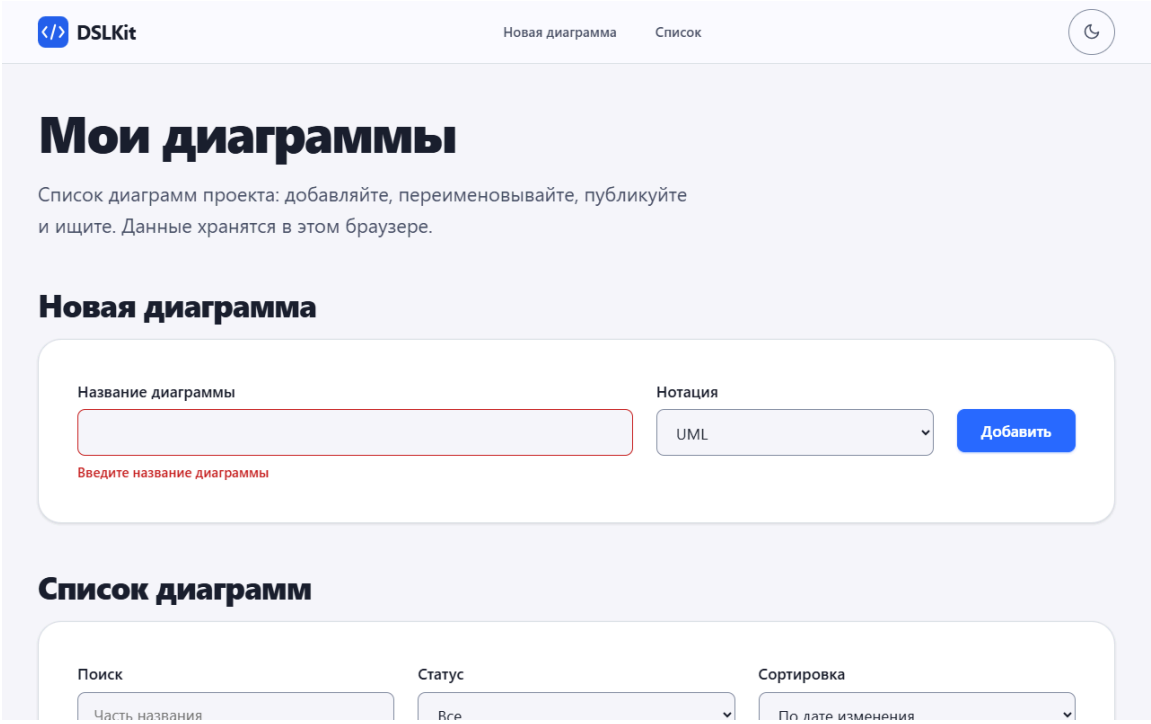


Рисунок 2 – Сообщение «Введите название диаграммы» под полем с красной рамкой

Новая диаграмма появилась первой в списке, а в строке статистики видно число записей на рисунке 3.

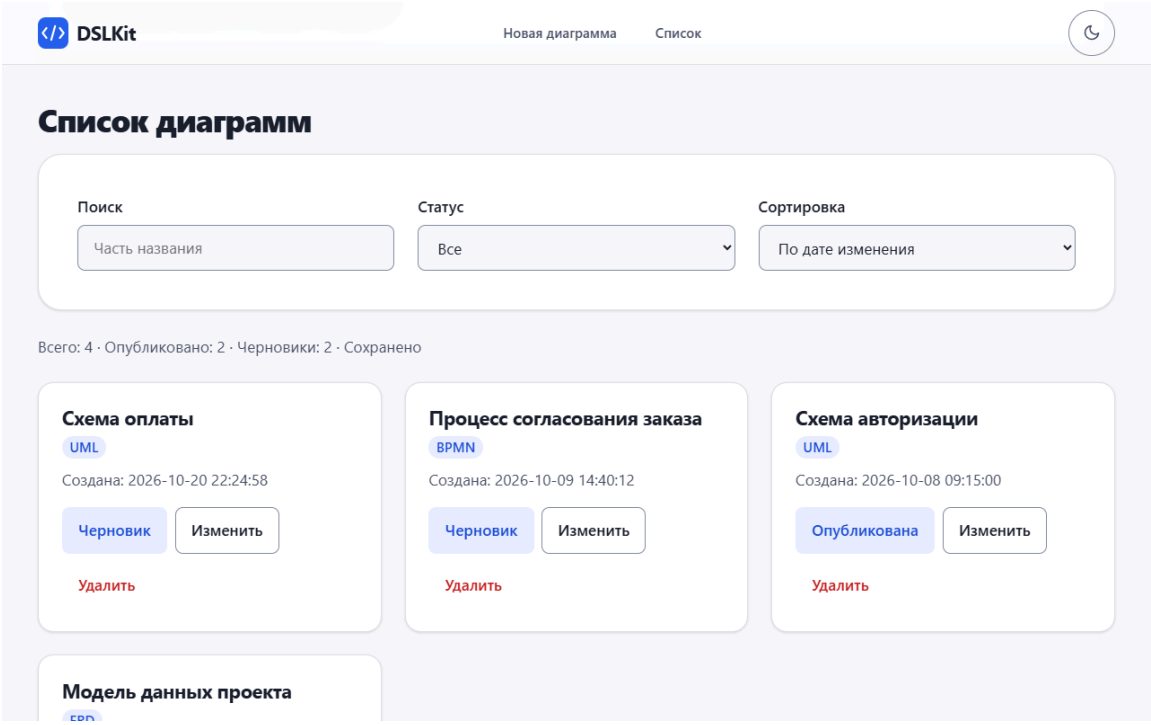


Рисунок 3 – Список диаграмм с новой записью и строкой статистики

Сразу после добавления записи строка статистики показывает «Сохранение…», как на рисунке 4. Через 500 мс тишины надпись меняется на «Сохранено».

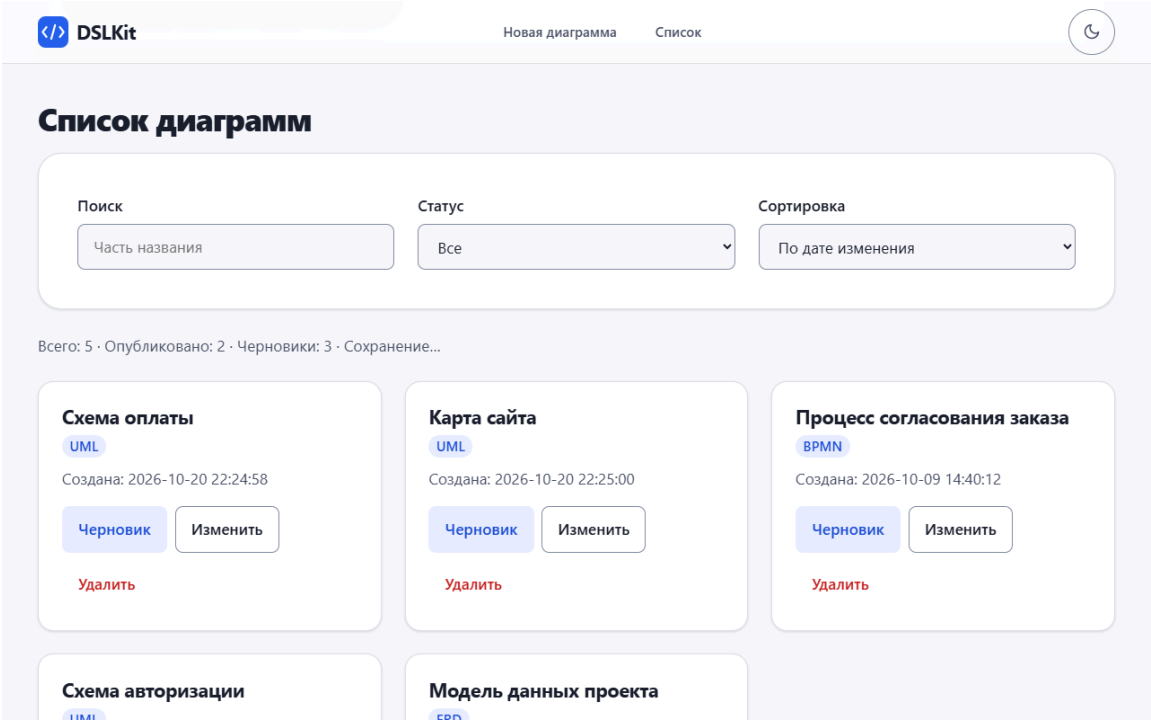


Рисунок 4 – Строка статистики с пометкой «Сохранение…» сразу после изменения

Фильтр по статусу оставляет в списке только опубликованные диаграммы, что показано на рисунке 5.

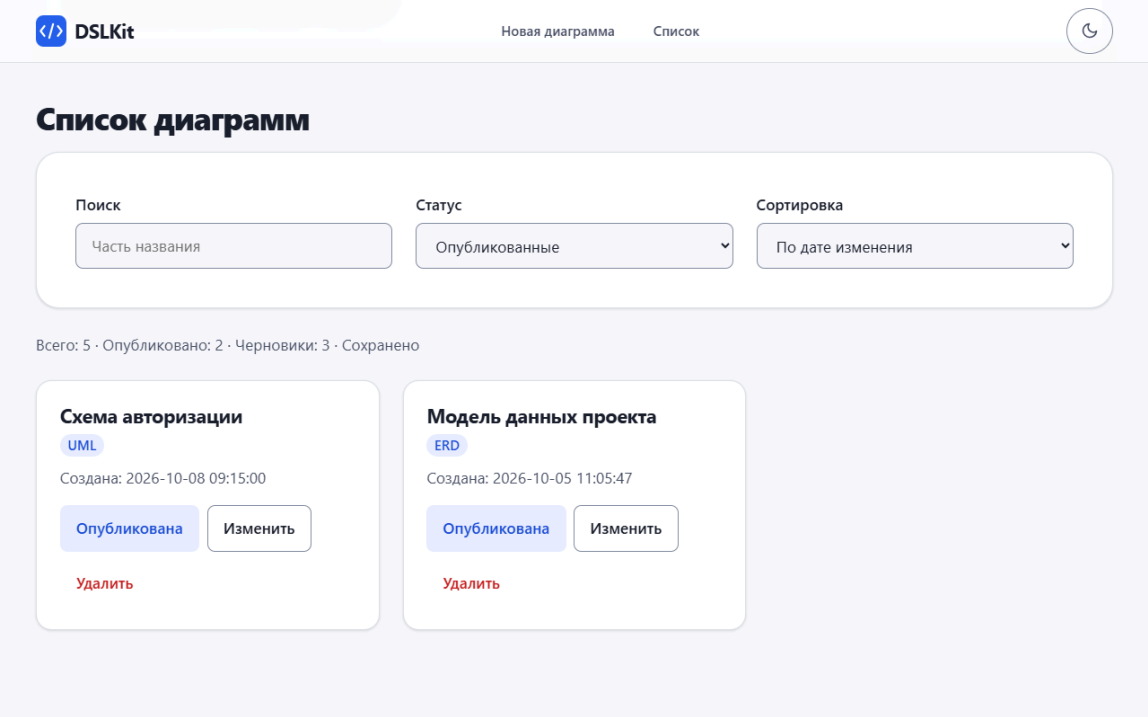


Рисунок 5 – Список после выбора статуса «Опубликованные»

Выбор сортировки по названию переставляет карточки по алфавиту, как на рисунке 6.

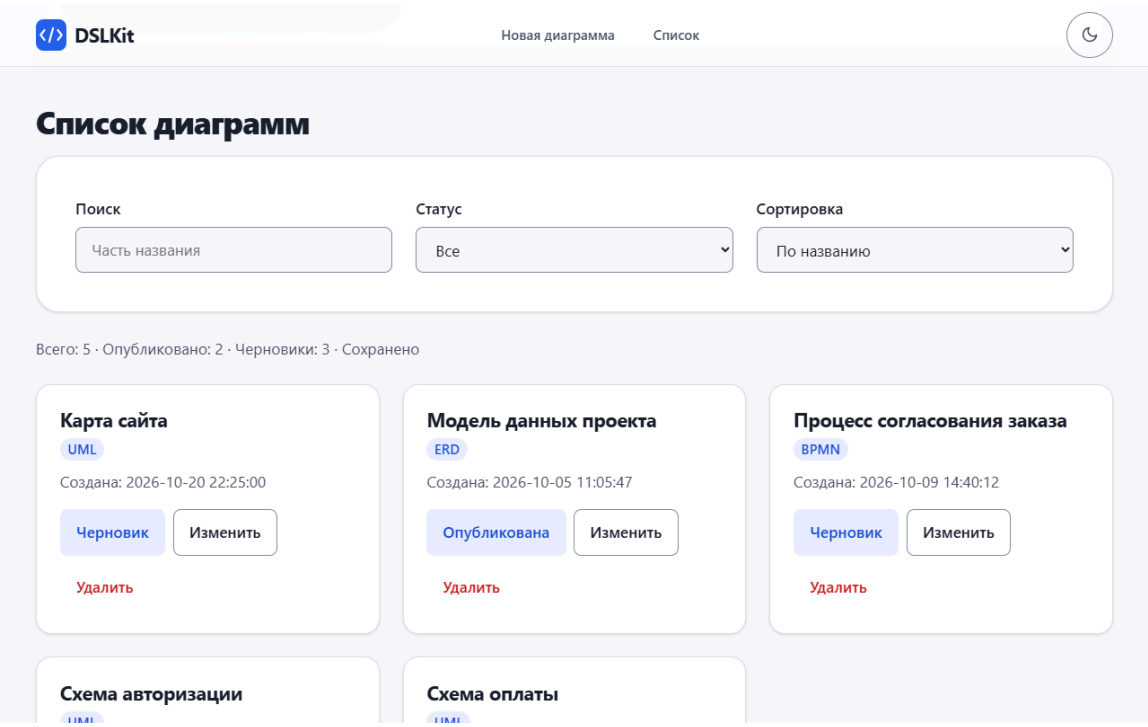


Рисунок 6 – Список диаграмм при сортировке «По названию»

Поиск по части названия оставляет подходящие записи, результат виден на рисунке 7.

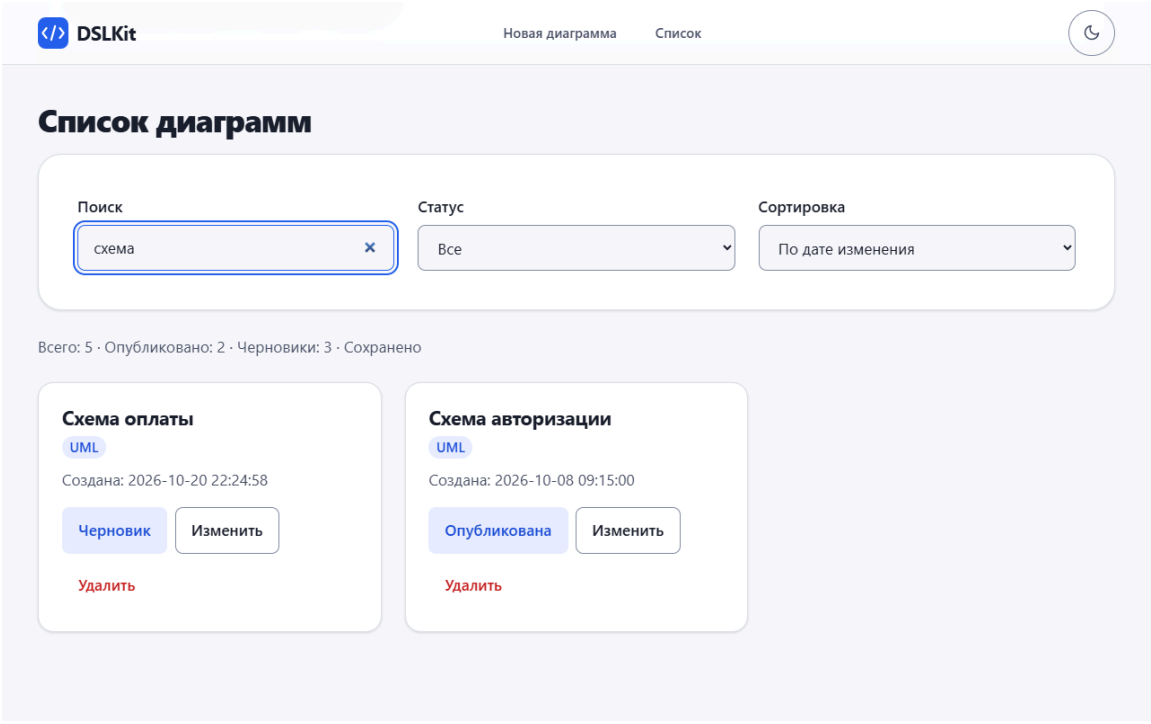


Рисунок 7 – Список диаграмм после ввода слова «схема» в поле поиска

Кнопка «Изменить» превращает название в поле ввода, а вместо неё появляется кнопка «Сохранить», как на рисунке 8.

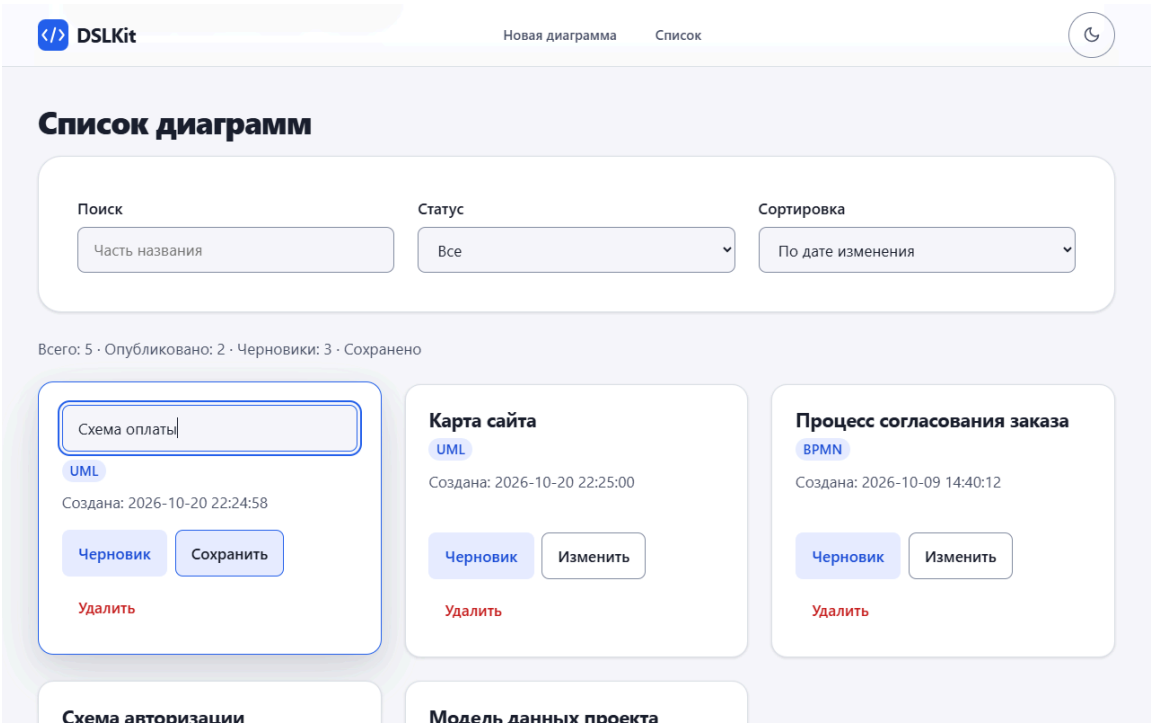


Рисунок 8 – Карточка диаграммы в режиме редактирования названия

Если стереть название и нажать Enter, под полем появляется сообщение, а запись не меняется, что видно на рисунке 9. Клавиша Esc отменяет правку.

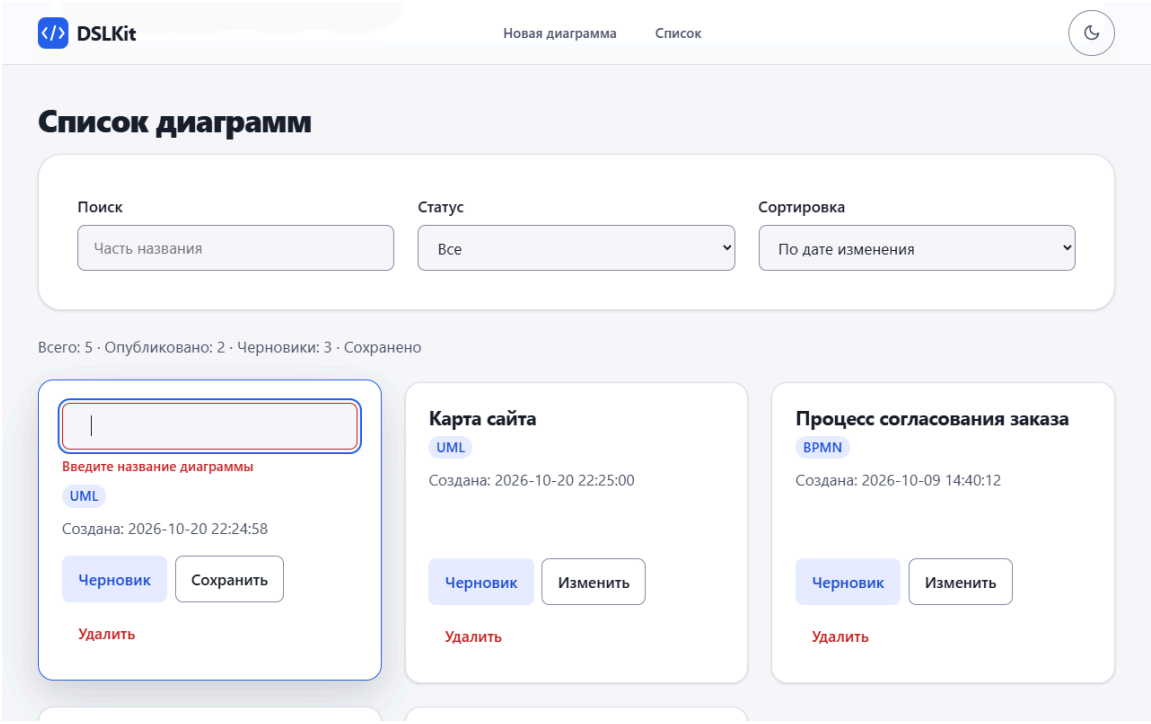


Рисунок 9 – Сообщение об ошибке под полем редактирования пустого названия

После нажатия «Удалить» запись исчезает из списка и из статистики, результат показан на рисунке 10.

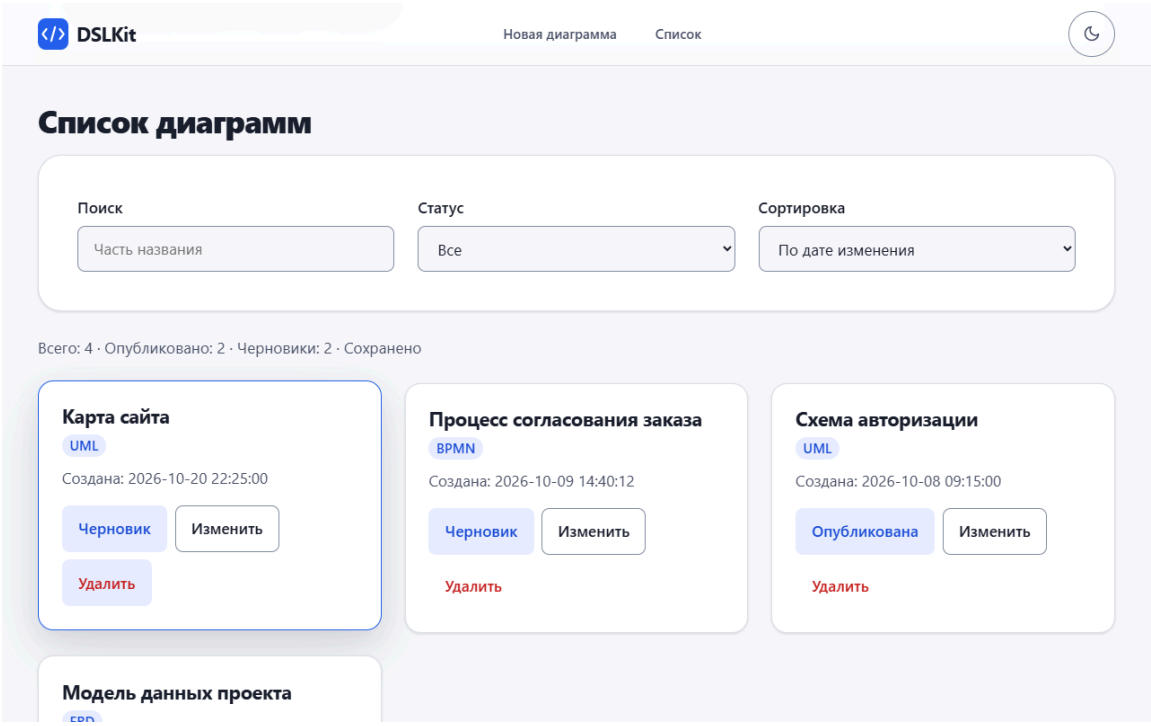


Рисунок 10 – Список диаграмм после удаления одной записи

После перезагрузки страницы список остаётся прежним, что видно на рисунке 11.

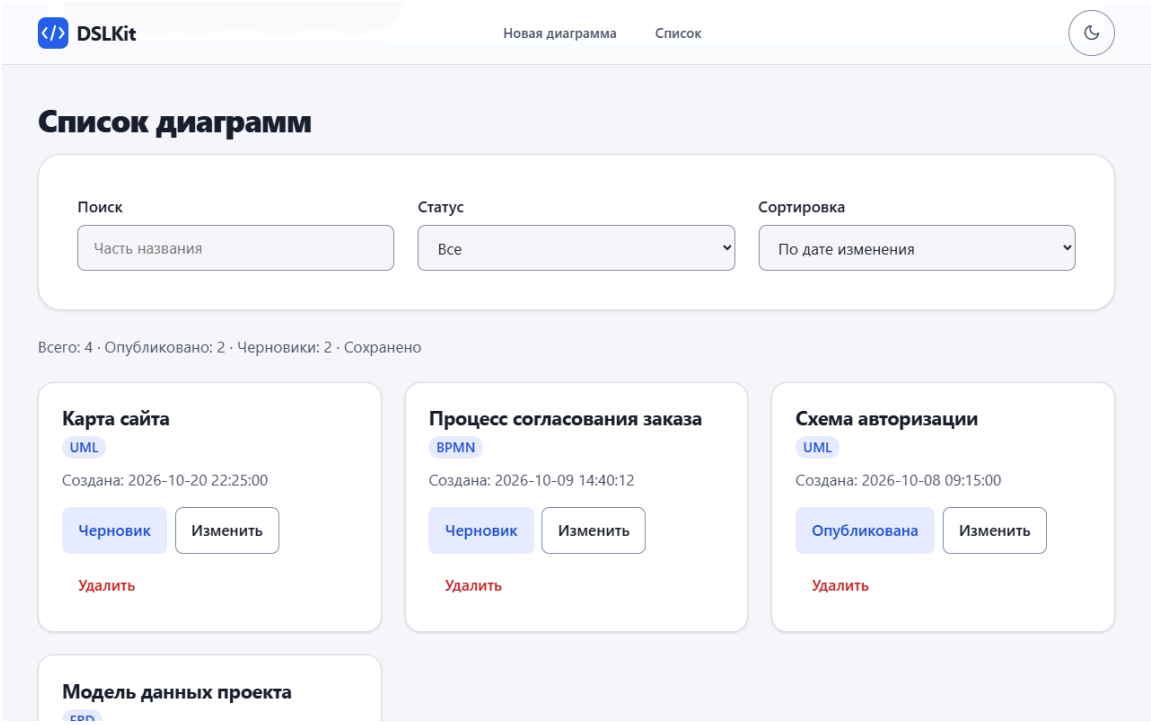


Рисунок 11 – Список диаграмм после перезагрузки страницы

Если в списке нет записей, вместо карточек выводится сообщение, как на рисунке 12.

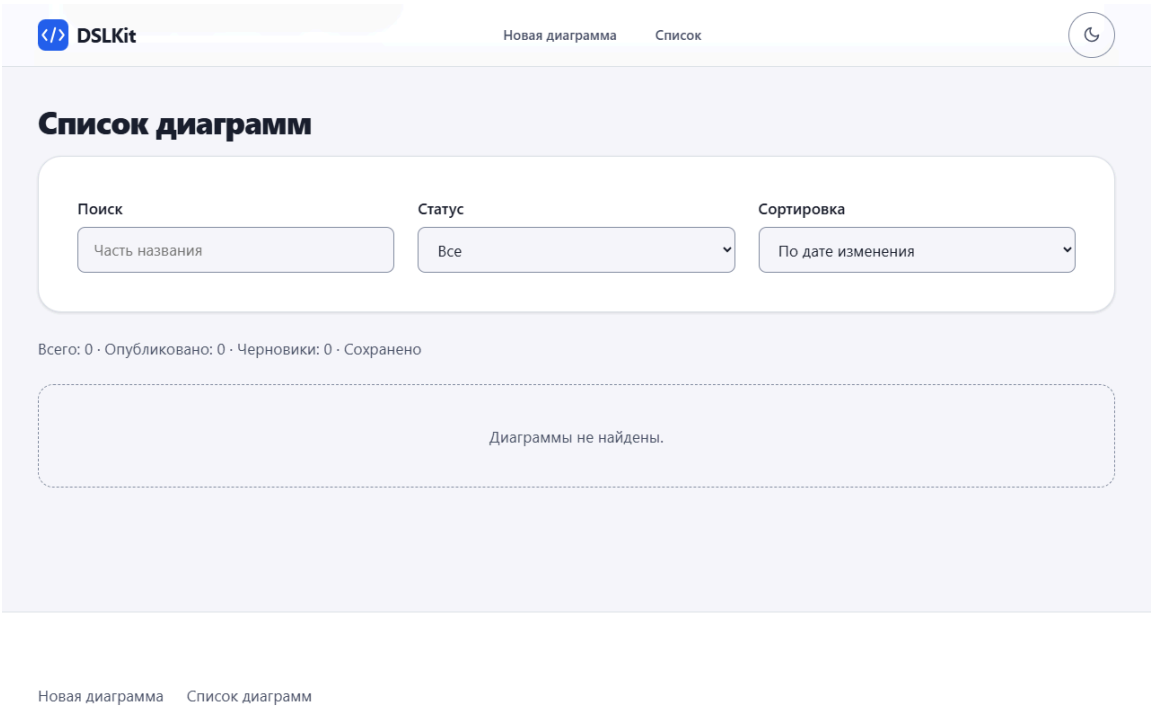


Рисунок 12 – Сообщение «Диаграммы не найдены» при пустом списке

На ширине телефона 375 пикселей раскладка переходит в одну колонку, как на рисунке 13.

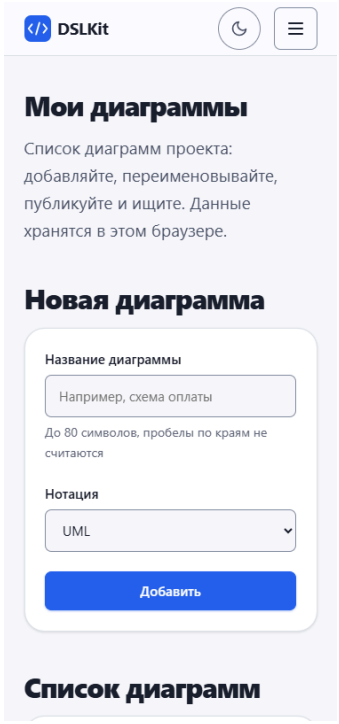


Рисунок 13 – Страница приложения на экране шириной 375 пикселей

Меню на телефоне открывается кнопкой с тремя полосками и показано на рисунке 14.

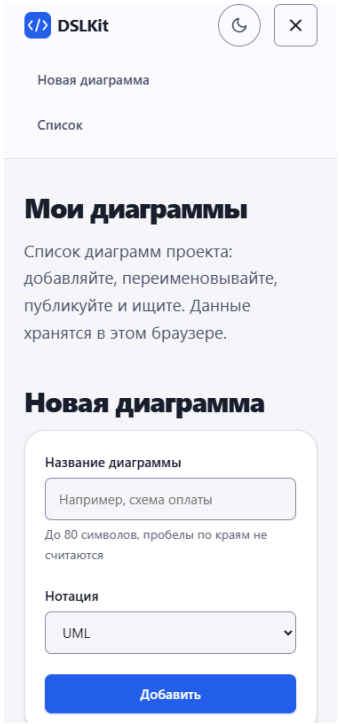


Рисунок 14 – Раскрытое меню на экране шириной 375 пикселей

Тёмная тема включается значком луны в шапке и показана на рисунке 15.

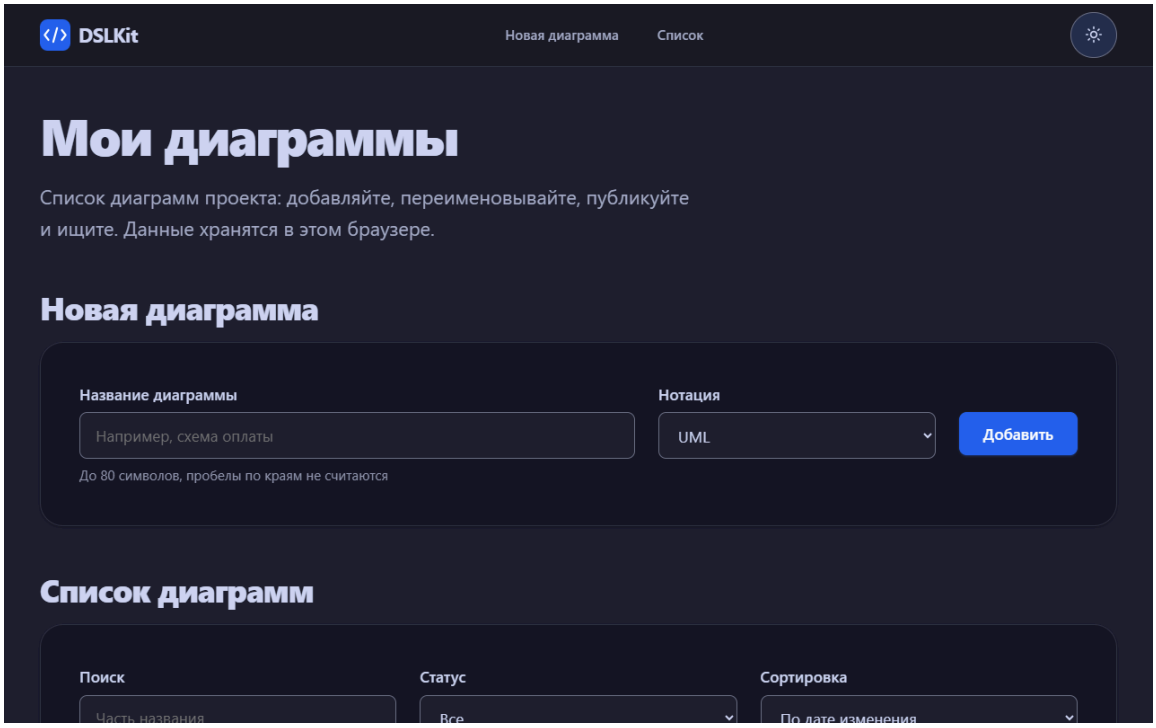


Рисунок 15 – Страница приложения в тёмной теме

3.9 Тестирование

В этой работе нет сервера и маршрутов, поэтому страницы 401, 403 и 404 не нужны, а для ошибок самого приложения есть две страницы, построенные из каталога текстов `src/shell/errorCatalog.js`. Страница 500 показывается, когда при отрисовке возникла непредвиденная ошибка, а страница «Нет связи с сервером» с кнопкой повтора показывается, когда не удалось скачать часть кода. Каждая отвечает на три вопроса: что это за ошибка, почему она возникла и как её исправить. Технический текст ошибки уходит только в консоль и на страницу не попадает.

Сто автоматических тестов проверяют чистые функции, оба хука и поведение приложения: добавление, удаление, редактирование с Enter и Esc, переключение статуса, фильтры, сортировку, восстановление списка, проверку пробельных и огромных значений, уникальное время создания, повреждённые данные хранилища, вывод разметки как текста, переключатель темы и страницу 500. Отдельный тест проверяет имитацию загрузки за 1 секунду с индикатором на поддельных таймерах. Ручная проверка описана в `README.md`, содержимое `localStorage` смотрится во вкладке Application инструментов разработчика.

ЗАКЛЮЧЕНИЕ

Приложение на Vite реализует добавление, удаление и редактирование диаграмм, фильтрацию и сортировку, сохранение в `localStorage` и три дополнительных эффекта, заголовок страницы, автосохранение с debounce и имитацию загрузки с сервера за 1 секунду. Логика хранения вынесена в пользовательский хук, что упростит замену `localStorage` на REST API в лабораторной работе №5.

ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

9. **Как сохранить объект или массив в `localStorage`?** Сериализовать через `JSON.stringify`, а прочитать через `JSON.parse` с `try/catch` и проверкой формы данных.
10. **Что будет, если изменить состояние сразу после `setState`?** Значение в текущем вызове не изменится, потому что обновления группируются и применяются на следующей отрисовке. Для цепочки используют функциональную форму.
11. **Как имитировать загрузку с сервера?** В `useEffect` запустить `setTimeout`, который через секунду записывает данные в состояние, а в очистке вызвать `clearTimeout`.

ПРИЛОЖЕНИЕ А

(обязательное)

Ключевые части кода

```
// hooks/useLocalStorage.js: загрузка через инициализатор, запис
ь эффектом
export function useLocalStorage(key, initialValue) {
  const [value, setValue] = useState(() => readStorage(key, init
ialValue));
  useEffect(() => {
    localStorage.setItem(key, JSON.stringify(value));
  }, [key, value]);
  return [value, setValue];
}

// hooks/useAutosaveStatus.js: debounce 500 мс с очисткой таймер
а
useEffect(() => {
  setStatus('saving');
  const timer = setTimeout(() => setStatus('saved'), AUTOSAVE_DE
LAY);
  return () => clearTimeout(timer);
}, [value]);

// hooks/useServerLoad.js: имитация загрузки с сервера, 1 секунд
а
useEffect(() => {
  if (!loading) return undefined;
  const timer = setTimeout(() => setLoading(false), loadConfig.d
elay);
  return () => clearTimeout(timer);
}, [loading]);

// App.jsx: функциональные обновления состояния и заголовок стра
ницы
const deleteDiagram = (id) => setDiagrams((prev) => prev.filter
((d) => d.id !== id));

useEffect(() => {
  document.title = `DSLKit: диаграмм ${diagrams.length}`;
}, [diagrams.length]);
```